

KTU DATA STRUCTURES & ALGORITHMS

Restaurant Management System

A C-language case study in Stacks, Queues, Linked Lists,
Binary Search Trees, and Merge Sort

Stack

Queue

Linked List

BST

Merge Sort

Why This Project Matters for DSA

A restaurant runs on real operations — taking orders, cooking them in sequence, billing, and reviewing sales. Each operation naturally maps to a classic data structure, which is exactly what this mini-project demonstrates in working C code.

- **Concept → Code**

Every abstract DS taught in class is implemented and exercised through a real menu-driven program.

- **End-to-end flow**

Order placement, kitchen service, cancellation, history, and billing form one connected pipeline.

- **Multiple structures, one system**

Shows how different structures cooperate instead of being studied in isolation.

Data Structures Used

- S** **Stack**
Undo last order (LIFO)
- Q** **Queue**
Kitchen order handling (FIFO)
- L** **Linked List**
Billing history log
- B** **Binary Search Tree**
Menu sorted by price
- M** **Merge Sort**
Sort served bills by total

The Two Core Structures: MenuItem & Order

Before any algorithm runs, the program needs a way to represent data. Two C structs form the foundation that every module below operates on.

struct MenuItem

id	unique item number
name[30]	dish name (string)
price	cost of the dish

struct Order

id	unique order number
customer[30]	customer name
items[5]	array of MenuItem (composition)
itemCount	no. of items ordered
total	sum of item prices

Key idea: an Order is built from MenuItem objects — this composition is what every data structure below stores and moves around.

Stack — Undo the Last Order

A Stack follows Last-In-First-Out (LIFO): the most recently placed order is the first one that can be cancelled.

```
typedef struct {
    Order data[MAX];
    int top;
} Stack;

void push(Stack *s, Order order) {
    if (s->top < MAX - 1)
        s->data[++s->top] = order;
}

int pop(Stack *s, Order *order) {
    if (s->top == -1) return 0;
    *order = s->data[s->top--];
    return 1;
}
```

push() Places a new order on top — array index 'top' increments first.

pop() Removes and returns the top order — that's the “undo” action.

Complexity $O(1)$ for both push and pop — no shifting of elements needed.

Stack of Orders



Queue — Kitchen Order Processing

A Queue follows First-In-First-Out (FIFO): the kitchen must cook orders in the exact sequence customers placed them — fairness matters here.

```
typedef struct {
    Order data[MAX];
    int front, rear;
} Queue;

void enqueue(Queue *q, Order order) {
    if (q->rear < MAX - 1)
        q->data[++q->rear] = order;
}

int dequeue(Queue *q, Order *order) {
    if (q->front > q->rear) return 0;
    *order = q->data[q->front++];
    return 1;
}
```

enqueue() New order joins at the 'rear' — placed at the back of the kitchen line.

dequeue() Order is served/removed from the 'front' — oldest order first.

Complexity O(1) per operation using front/rear index pointers.

Queue of Orders

FRONT (served next)

REAR (new orders join)



Note: Stack and Queue run side-by-side on the same order — placing an order pushes it AND enqueues it, serving two different customer needs (cancel vs. cook).

Linked List — Billing History Log

Once an order is served, it must be permanently recorded. A singly linked list stores this growing, unbounded history — unlike arrays, it needs no fixed size.

```
typedef struct Node {
    Order order;
    struct Node *next;
} Node;

Node *head = NULL;

void addHistory(Order order) {
    Node *n = malloc(sizeof(Node));
    n->order = order; n->next = NULL;
    if (head == NULL) head = n;
    else {
        Node *t = head;
        while (t->next) t = t->next;
        t->next = n;
    }
}
```

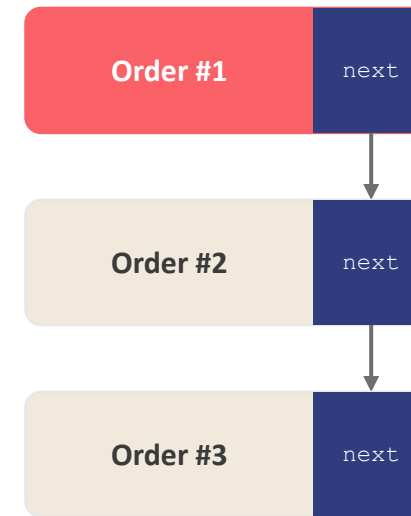
Insertion

Traverses to the last node, then attaches the new node — preserves service order.

Complexity

$O(n)$ per insert here (tail pointer not cached); $O(n)$ space overall for n orders.

head → Order Chain → NULL



NULL

Binary Search Tree — Menu Sorted by Price

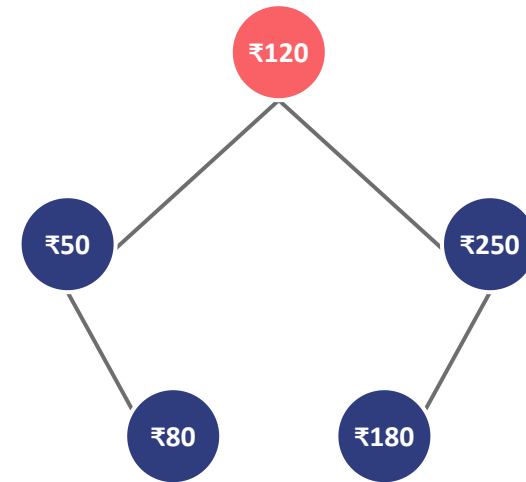
Every menu item is inserted into a BST keyed on price. An in-order traversal then prints the entire menu from cheapest to costliest — automatically sorted, no separate sort step needed.

```
BSTNode* insertBST(BSTNode *root, MenuItem item) {
    if (root == NULL) return createNode(item);
    if (item.price < root->item.price)
        root->left = insertBST(root->left, item);
    else
        root->right = insertBST(root->right, item);
    return root;
}

// In-order traversal → sorted by price
void displayMenu(BSTNode *root) {
    if (root) {
        displayMenu(root->left);
        print(root->item);
        displayMenu(root->right);
    }
}
```

Complexity: insert & search average $O(\log n)$, worst case $O(n)$ for a skewed tree; in-order traversal is always $O(n)$.

Resulting Tree (insert order: 120, 250, 50, 180, 80)



In-order visit → ₹50, ₹80, ₹120, ₹180, ₹250 (cheapest to costliest)

Merge Sort — Ranking Bills by Total

Before printing served bills, the array is sorted by total amount using Merge Sort — chosen for its guaranteed $O(n \log n)$ performance regardless of input order.

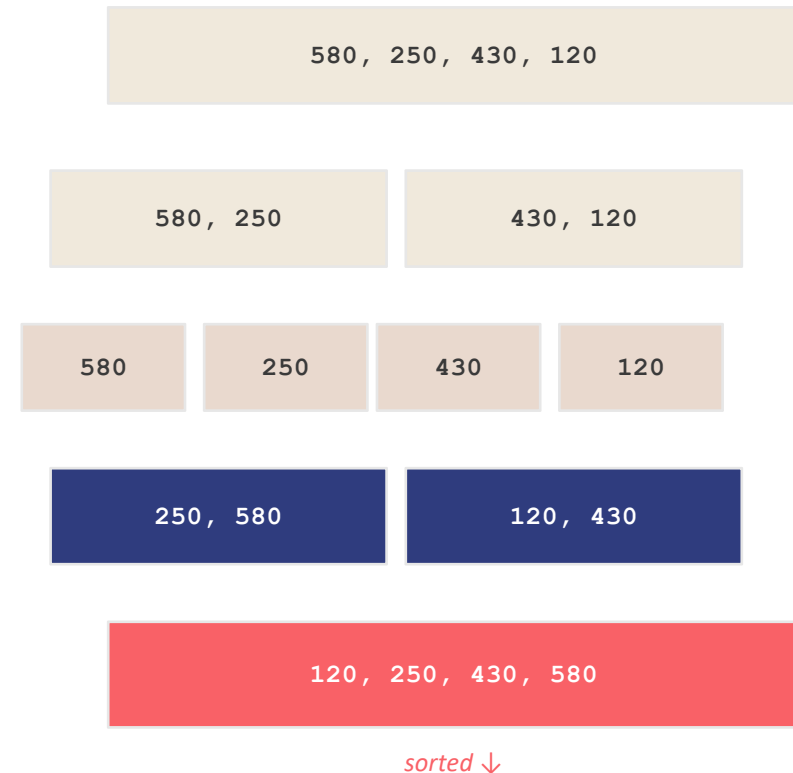
```
void mergeSort(Order arr[], int l, int r) {  
    if (l < r) {  
        int mid = (l + r) / 2;  
        mergeSort(arr, l, mid);  
        mergeSort(arr, mid+1, r);  
        merge(arr, l, mid, r);  
    }  
}  
  
// merge(): compares arr[i].total vs arr[j].total  
// and writes the smaller total back first
```

Divide Array is recursively split at the midpoint until each part has 1 element.

Conquer merge() combines two sorted halves into one sorted range by total.

Complexity $O(n \log n)$ time in every case; $O(n)$ extra space for the merge buffer.

Divide → Sort → Merge



How the Modules Connect: Main Program Loop

main() runs a menu-driven infinite loop. Every choice routes to exactly one module, and several choices touch more than one structure at once.

1

View Menu

`displayMenu(root)` – BST in-order traversal

2

Place Order

`enqueue()` + `push()` – Queue & Stack both updated

3

Serve Next Order

`dequeue()` then `addHistory()` – Queue → Linked List

4

Undo Last Order

`pop()` – Stack removes most recent order

5

View Sorted Bills

copy history → array → `mergeSort()` by total

0

Exit

break out of the `while(1)` loop

Time & Space Complexity Summary

Data Structure	Key Operation	Time Complexity	Space	Used For
Stack (array)	push / pop	$O(1)$	$O(n)$	Undo last order
Queue (array)	enqueue / dequeue	$O(1)$	$O(n)$	Kitchen order line
Linked List	insert at tail	$O(n)$	$O(n)$	Billing history
Binary Search Tree	insert / search	$O(\log n)$ avg, $O(n)$ worst	$O(n)$	Menu sorted by price
Merge Sort	sort n bills	$O(n \log n)$	$O(n)$	Rank bills by total

Takeaway: constant-time structures (Stack, Queue) handle live, high-frequency actions; the Linked List favors flexible growth over speed; the BST keeps the menu ordered automatically; Merge Sort guarantees predictable $O(n \log n)$ ranking however the bills arrive.

Why Each Structure Fits Its Role

1 Stack → Undo

Cancellations almost always target the most recent order, which is precisely what LIFO retrieves first — no searching required.

2 Queue → Kitchen

Fair service demands strict arrival order. FIFO guarantees the first customer to order is the first to be served.

3 Linked List → History

The number of completed orders is unknown and unbounded in advance, so dynamic node allocation beats a fixed-size array.

4 BST → Menu

Storing items by price keeps the menu perpetually sorted; an in-order walk prints it without a separate sorting pass.

5 Merge Sort → Bills

A stable, guaranteed $O(n \log n)$ sort is needed once, on demand, for reporting — recursion mirrors the problem naturally.

One System, Five Structures, One Lesson

This project shows that data structures are not academic abstractions — they are the natural vocabulary for describing real workflows. A restaurant's everyday actions (cancel, cook, bill, browse, rank) each found their cleanest expression in a specific structure, taught to choose based on the access pattern the problem demands, not habit.

- Match the structure to the access pattern (LIFO, FIFO, ordered, dynamic).
- Combine structures — one order can live in a Queue, a Stack, and later a Linked List.
- Recursion (BST insert, Merge Sort) elegantly expresses divide-and-conquer logic.

Possible Extensions

- Hash table for $O(1)$ menu-item lookup by ID
- Priority queue for VIP / urgent orders
- AVL / self-balancing tree to avoid BST skew
- File storage so billing history persists
- Graph-based table/seating allocation

Thank You